Strings

CSCI 1101

2024-02-01

Text

One kind of information that we want to represent in our programs is text.

Python represents text using the type of **strings**, written `str`.

We represent strings by writing text between quotes, either single or double:

```
size = 'large'

color = "blue"
```

The kind of quote doesn't matter, except if your string contains one kind then it's convenient to use the other kind:

```
line_1 = "don't ever"

line_2 = 'say "never"'
```

Escape Characters

It's also possible to use the same kind of quote in a string by **escaping** it with a backslash:

```
line_3 = "never say \"never\""
line_4 = 'don\'t say it!'
```

Besides the quotes there are a few other useful **escaped characters**:

```
\t              #  insert tab
\n              #  insert newline
\\              #  insert backslash
\<newline>      #  ignore newline
```

Working with Strings

Some of the operators we met last week are *overloaded* to work with strings:

```
"Bow" + "doin"       #  string concatenation
"Bowdoin!" * 3       #  string repetition
```

We can get a string representation of a number using the `str` function:

```
str(2 + 2)
str(5 / 2)
```

And we can combine these to make compound string expressions:

```
"I have " + str(1 + 2) + " apples!"
```

Multiline Strings

To write a multiline string without "\n" you can use a triple-quoted string, using either of the allowed quote types (this is what a function *docstring* is).

```
two_cities = """\
It was the best of times,
it was the worst of times,
it was the age of wisdom,
it was the age of foolishness,
it was the epoch of belief,
it was the epoch of incredulity,
it was the season of Light,
it was the season of Darkness...
"""
```

But if we ask python to evaluate this, it doesn't look the way we might expect.

Printing Strings

Python has a function called `print` that displays strings in a nice way.

```
print('hello\nworld')
```

```
print(two_cities)
```

Greeting the User

Using the `print` function we can write a function that greets the user by name:

```
def greet(name) :
    print("Hello " + name + "!")
```

Where is the `return` statement?

None and NoneType

Python has a type called `NoneType` that has only one value, called `None`.

By convention, we use `None` to indicate that a function returns “nothing interesting”.

If we omit the `return` statement in a function definition,
then python automatically inserts an implicit “`return None`” for us.

Returning vs Printing

It is important to understand the difference between *returning* and *printing* a result.

Returning a result gives that result to the caller of the function,
so that it can be used in the rest of the program.

Printing a result causes it to be displayed on the computer screen,
but does *not* return it to the function's caller.
This means that it is "lost" to the rest of the program.

The `print` function has signature `(str) --> NoneType`.

Because `print` always returns `None`, your program doesn't have access to what was `print`ed.

Returning vs Printing in the Interactive Interpreter

The interactive interpreter is built so that that each time it evaluates an expression to a value other than `None`, it displays a string representation of it.

You can also ask the interpreter to display a string using the `print` function.

Although these two behaviors look superficially similar, it is important to be able to distinguish them.

```
'bow' + 'doin'            #  evaluates to 'bowdoin'

print('bow' + 'doin')     #  evaluates to None
```

I wish it would use colors or fonts to distinguish these behaviors more clearly, but it does not.

Program Effects

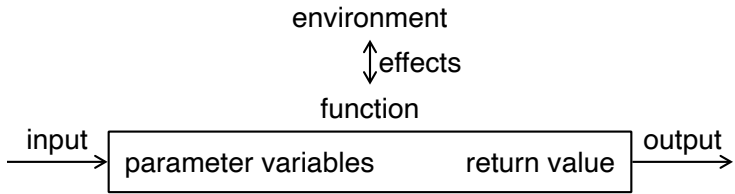
Printing something to the screen is an example of a program effect.

An **effect** is an interaction that a program makes with its environment in the process of computing its result.

A function acts like an *expression* because it evaluates to its return value.

A function also acts like a *statement* because it can perform effects.

We can visualize this flow of information as follows:



Activity: Returning vs Printing

Consider the following similar-looking functions:

```
def double(num) :
    return 2 * num
```

```
def print_double(num) :
    print(str(2 * num))
```

Predict the result (value and effects) of evaluating the following expressions:

```
double(2)
```

```
print_double(2)
```

```
type(double(2))
```

```
type(print_double(2))
```

```
double(double(2))
```

```
print_double(print_double(2))
```

Moral: returning is more flexible that printing because it is *composable*.

Getting User Input

The `print` function lets us display information to the user's screen.

There is another function `input : (str) --> str`
that gets information from the user's keyboard.

Its argument string, called a **prompt**, should explain what information we want.

```
def meet() :
    """ signature: () --> str """
    return input("What's your name? ")
```

Notice that this function takes zero arguments.
It receives its information from its environment as an *effect*.

Activity: Meet and Greet

Use the `meet` and `greet` functions to write a function,

```
def meet_and_greet() :
    """ signature: () --> NoneType """
```

that asks the user's name and then greets them by their name.

For example:

```
meet_and_greet()
What's your name? Bond, James Bond
Hello Bond, James Bond!
```

Type Conversion Functions

Today we met the function `str` that tries to convert its argument into a string.

There are similar functions for converting data to the other types we have met.

The function `int` tries to convert its argument into an integer.

The function `float` tries to convert its argument into a floating-point number.

These functions may fail, or may give unexpected results:

```
int(3.999999999)
float("1/3")
```

When you call some functions with an argument of the wrong type python will silently apply one of these functions to try to convert it to the right type.

This can cause subtle errors unless you deeply understand automatic conversion. It's better to not rely on this and instead respect the type specifications.

Activity: Parsing Strings

Still, these functions are useful for extracting numerical data from user input.
This is called **parsing**.

Consider the following program that tries to add two numbers provided by the user:

```
def adder() :
    num1 = input("What is the first number? ")
    num2 = input("What is the second number? ")
    print("The sum is " + (num1 + num2))
```

It does not work as expected.

Add type conversion functions to make it work properly.

To Do

Finish *Homework 2: Strings* on CodeRunner by 11:00AM next Thursday.